Practical 13: Web Application using Flask Framework

Aim

To study and implement a simple web application using the Flask framework in Python.

Theory

Flask is a lightweight, minimalistic Python 'micro' web framework built on top of the Werkzeug WSGI toolkit and Jinja2 templating engine, used to build web applications and RESTful APIs with minimal boilerplate compared to full-stack frameworks like Django. A Flask application is represented by an instance of the Flask class, typically created with app = Flask(__name__), where __name__ helps Flask locate resources relative to the module. Route decorators (@app.route(path)) bind a URL path (and optionally HTTP methods like GET/POST) to a Python function known as a view function; when a client requests that path, Flask invokes the corresponding view function and sends whatever it returns (a string, JSON, or rendered HTML template) back as the HTTP response body. Multiple routes allow a single application to serve different pages or API endpoints, as demonstrated by the '/' and '/about' routes. The app.run() method starts Flask's built-in development server (not intended for production use) to listen for and serve incoming HTTP requests on a local host and port, with debug=True enabling auto-reload on code changes and detailed in-browser error tracebacks during development.

Algorithm

1. Import Flask and create a Flask application instance.
2. Define a route '/' mapped to a home() function that returns a welcome message.
3. Define a route '/about' mapped to an about() function that returns an about message.
4. Run the Flask application using app.run() when the script is executed directly.
5. Send GET requests to '/' and '/about' and verify the responses returned by the server.

Program

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome to my Flask Web App!"

@app.route("/about")
def about():
    return "This is the About page."

if __name__ == "__main__":
    app.run(debug=True)
```

Output

```
Running: python p13.py
 * Running on http://127.0.0.1:5000 (Press CTRL+C to quit)

Tested using Flask's test client:
GET / -> 200  "Welcome to my Flask Web App!"
GET /about -> 200  "This is the About page."
```

Result / Conclusion

A simple Flask web application with multiple routes was successfully created and its endpoints were verified to return the correct responses.

Viva Questions

Q1. What does the @app.route() decorator do in Flask?

Ans: @app.route('/path') is a Python decorator that registers the decorated function as the handler (view function) for the specified URL path. When a client sends an HTTP request matching that path, Flask's URL dispatcher (Werkzeug's routing system) calls the corresponding view function and returns its response. By default, only GET requests are accepted; add methods=['GET','POST'] to handle multiple HTTP methods.

Q2. What is the difference between Flask and Django?

Ans: Flask is a micro-framework: minimal built-in features (routing, templating, development server), highly flexible, you add only what you need (ORM, auth, etc. are separate packages). Django is a full-stack framework: batteries-included with ORM, admin panel, authentication, forms, migrations out of the box. Flask is better for small apps, microservices, and REST APIs. Django is better for large, database-heavy web applications where convention over configuration is preferred.

Q3. What is WSGI and why is it important for Flask?

Ans: WSGI (Web Server Gateway Interface) is a Python standard (PEP 3333) that defines how web servers communicate with Python web applications. Flask is built on Werkzeug, a WSGI toolkit. A WSGI server (Gunicorn, uWSGI) calls the Flask app as a Python callable, passing the request environment and a response callback. This standard interface allows Flask apps to run on any WSGI-compatible server in production, decoupling the framework from the server.

Q4. Why should debug=True never be used in production?

Ans: debug=True enables: (1) automatic server restart on code changes; (2) the Werkzeug interactive debugger — an in-browser Python REPL that appears on unhandled exceptions. The interactive debugger allows arbitrary Python code execution in the server's process, which is a critical security vulnerability. Any visitor who triggers an error page could execute system commands. In production, use debug=False and a WSGI server like Gunicorn with proper error logging.

Q5. How does Flask's if __name__ == '__main__' guard work?

Ans: When Python runs a file directly (python app.py), __name__ is set to '__main__'. When the file is imported as a module (import app), __name__ is the module name ('app'). The guard ensures app.run() is only called when the script is executed directly, not when imported (e.g., by a WSGI server or test suite that imports the app object without wanting to start the server).